

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Example 5.2 1D Decision boundary for the Iris dataset
5  Machine Learning for Engineering Problem Solving
6  @author: Austin R.J. Downey
7  """
8
9  import IPython as IP
10 IP.get_ipython().run_line_magic('reset', '-sf')
11
12 import numpy as np
13 import matplotlib.pyplot as plt
14 import sklearn as sk
15
16
17 cc = plt.rcParams['axes.prop_cycle'].by_key()['color']
18 plt.close('all')
19
20
21 %% Load your data
22
23 # We will use the Iris data set. This dataset was created by biologist Ronald
24 # Fisher in his 1936 paper "The use of multiple measurements in taxonomic
25 # problems" as an example of linear discriminant analysis
26
27 iris = sk.datasets.load_iris()
28
29 # for simplicity, extract some of the data sets
30 X = iris['data'] # this contains the length of the pedals and sepals
31 Y = iris['target'] # contains what type of flower it is
32 Y_names = iris['target_names'] # contains the name that aligns with the type of the
    flower
33 feature_names = iris['feature_names'] # the names of the features
34
35 # plot the Sepal data
36 plt.figure(figsize=(6.5,3))
37 plt.subplot(121)
38 plt.grid(True)
39 plt.scatter(X[Y==0,0],X[Y==0,1],marker='o')
40 plt.scatter(X[Y==1,0],X[Y==1,1],marker='s')
41 plt.scatter(X[Y==2,0],X[Y==2,1],marker='d')
42 plt.xlabel(feature_names[0])
43 plt.ylabel(feature_names[1])
44
45
46 plt.subplot(122)
47 plt.grid(True)
48 plt.scatter(X[Y==0,2],X[Y==0,3],marker='o',label=Y_names[0])
49 plt.scatter(X[Y==1,2],X[Y==1,3],marker='s',label=Y_names[1])
50 plt.scatter(X[Y==2,2],X[Y==2,3],marker='d',label=Y_names[2])
51 plt.xlabel(feature_names[2])
52 plt.ylabel(feature_names[3])
53 plt.legend(framealpha=1)
54 plt.tight_layout()
55
56
57 %% Train a Logistic Regression model
58
59 # define the features (X) and the output (Y)
60 X_pedal = iris["data"][:, 3:] # consider just the petal width
61 y_pedal = iris["target"] == 2 # 1 if Iris-Virginica, else 0
62
63 # Build the logistic Regression model and train it.
64 log_reg = sk.linear_model.LogisticRegression( C=1)
65 log_reg.fit(X_pedal, y_pedal)
66 # Note: The hyper-parameter controlling the regularization strength of a

```

```

67 # Scikit-Learn LogisticRegression model is not alpha (as in other linear models),
68 # but its inverse: C. The higher the value of C, the less the model is regularized.
69
70 # Build a range of the feature (X) to predict over. Here we just consider pedal width.
71 X_new = np.linspace(0, 3, 1000)
72 X_new = np.expand_dims(X_new, axis=1)
73
74 # Use the Logistic Regression Model to predict the pedal type based on pedal width
75 y_proba = log_reg.predict_proba(X_new)
76
77 # plot the probability plots
78 plt.figure(figsize=(6.5,3))
79 plt.grid(True)
80
81 # plot the data used for training, set at 0 and 1
82 plt.scatter(X[Y==0,3],np.zeros(50),marker='o',label=Y_names[0],zorder=10)
83 plt.scatter(X[Y==1,3],np.zeros(50),marker='s',label=Y_names[1],zorder=10)
84 plt.scatter(X[Y==2,3],np.ones(50),marker='d',label=Y_names[2],zorder=10)
85
86 # plot the probability
87 plt.plot(X_new,y_proba[:,0], '--',color=cc[3],label='Not Iris - Virgincia')
88 plt.plot(X_new,y_proba[:,1],color=cc[2], label='Iris - Virgincia')
89
90 # find and plot the 50% decision boundary
91 x_at_50 = X_new[np.argmax(np.abs(y_proba[:,0] - 0.5)))]
92 plt.vlines(x_at_50,0,1,color='k',linestyles=':',label='Decision Boundary')
93
94 plt.xlabel('pedal width (cm)')
95 plt.ylabel('probability')
96 plt.legend(framealpha=1)
97 plt.tight_layout()
98
99 # make predictions on the model trained on the data.
100 log_reg.predict([[1.7]])

```